



Collectors Avanzados en Java

Agrupar, particionar y unir resultados de Streams

En esta presentación exploraremos técnicas avanzadas para recolectar y transformar los resultados de nuestros Streams en Java, llevando nuestro código a un nuevo nivel de expresividad y funcionalidad.

Aprenderemos a organizar, clasificar y manipular datos de forma declarativa y elegante, continuando con lo aprendido sobre operaciones terminales básicas.

¿Qué son los Collectors avanzados?

Los Collectors nos permiten **recolectar resultados de Streams en estructuras más complejas**, yendo más allá de las operaciones básicas como `toList()` o `toSet()`.

Son herramientas poderosas para transformar flujos de datos en estructuras organizadas que facilitan su posterior procesamiento y análisis.

Los más utilizados:

- `groupingBy()` - Agrupa elementos por una clave
- `partitioningBy()` - Divide elementos en dos grupos
- `joining()` - Une elementos en un único String





groupBy(): Agrupando elementos

Permite **agrupar elementos** según una función de clasificación, creando un `Map` donde las claves son los resultados de la función y los valores son listas de elementos.

```
Map<Integer, List<Persona>> porEdad = personas.stream()
    .collect(Collectors.groupingBy(Persona::getEdad));
```

Entrada: Lista de personas

Ana (25), Luis (30), María (25), Carlos (40), Elena (30)

Salida: Map agrupado por edad

25 → [Ana, María]

30 → [Luis, Elena]

40 → [Carlos]

Esta técnica resulta extremadamente útil para crear resúmenes, estadísticas o preparar datos para presentarlos agrupados por categorías.

partitioningBy(): Dividiendo en dos grupos

Divide los elementos en **exactamente dos grupos** según una condición booleana, creando un `Map`:

```
Map<Boolean, List<Persona>> porMayoriaEdad = personas.stream()
    .collect(Collectors.partitioningBy(p -> p.getEdad() >= 18));
```

Esto genera un mapa con solo dos claves posibles: `true` y `false`.

Ventajas:

- Más eficiente que `groupingBy` para casos binarios
- Garantiza que ambos grupos existan en el resultado (incluso vacíos)
- Ideal para análisis de "cumple/no cumple" condición



true

Mayores de edad

[Ana, Luis, María, Carlos,
Elena]

false

Menores de edad

[Miguel, Laura]

joining(): Uniendo elementos en un String

Une los elementos de un Stream en un **único String**, con opciones para personalizar los delimitadores.

```
String nombres = personas.stream()
    .map(Persona::getNombre)
    .collect(Collectors.joining(", "));

// Resultado: "Ana, Luis, María, Carlos, Elena"
```



Stream de objetos

[Persona{Ana}, Persona{Luis}, ...]



Extracción

.map(Persona::getNombre)



Unión

.collect(joining(", "))



Resultado

"Ana, Luis, María, Carlos, Elena"

También podemos especificar prefijos y sufijos:

```
Collectors.joining(", ", "Nombres: [", "]")
// "Nombres: [Ana, Luis, María, Carlos, Elena]"
```

Ejemplo integrador: Procesando datos de personas

Datos de entrada:

```
List personas = List.of(
    new Persona("Ana", 25),
    new Persona("Luis", 30),
    new Persona("María", 25),
    new Persona("Carlos", 40),
    new Persona("Elena", 30),
    new Persona("Miguel", 17),
    new Persona("Laura", 16)
);
```

Objetivos:

- Agrupar personas por edad
- Separar mayores y menores de edad
- Obtener un listado de nombres

Solución con Collectors avanzados:

```
// Agrupación por edad
Map> porEdad =
    personas.stream()
        .collect(Collectors.groupingBy(
            Persona::getEdad));

// Separación por mayoría de edad
Map> porMayoriaEdad =
    personas.stream()
        .collect(Collectors.partitioningBy(
            p -> p.getEdad() >= 18));

// Listado de nombres
String nombres = personas.stream()
    .map(Persona::getNombre)
    .collect(Collectors.joining(", "));
```

Resumen y próximos pasos

groupingBy()

Agrupar elementos según una función de clasificación en un Map con múltiples categorías.

partitioningBy()

Divide elementos en exactamente dos grupos según una condición booleana.

joining()

Une elementos en un único String con delimitadores personalizables.

Beneficios de usar Collectors avanzados:

- Código más **declarativo** y legible
- Mayor **expresividad** en las transformaciones de datos
- Menos código, menos errores, más **mantenible**

Próximo módulo:

Exploraremos el tipo `Optional` en Java para gestionar valores potencialmente nulos de forma segura y expresiva.